



Industrial Internship Report on “Service Marketplace Platform”

Prepared by

Kekan Mahesh Dipak

Sinhgad College of Engineering

Executive Summary

This report documents the six-week Industrial Internship conducted through upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

The assigned project was the Service Marketplace Platform, a deployable multi-tenant web application that helps customers discover local service providers, compare offerings, schedule appointments, and start secure hosted payment checkout.

The solution was implemented as a responsive React and TypeScript frontend supported by an Express and TypeScript API. OpenAPI-generated client hooks and Zod validation keep the browser and server contracts aligned, while Clerk provides account authentication and Stripe provides hosted payment collection.

The completed first build demonstrates provider discovery, merchant onboarding, storefront and service management, customer order history, reviews, administrative foundations, responsive UI states, and API smoke validation. The report also records current limitations, including seeded in-memory data and the need for production persistence and post-payment notification workflows.

TABLE OF CONTENTS

1	Preface.....	3
2	Introduction.....	4
2.1	About UniConverge Technologies Pvt Ltd.....	4
2.2	About upskill Campus.....	8
2.3	Objective.....	9
2.4	Reference.....	9
2.5	Glossary.....	10
3	Problem Statement.....	11
4	Existing and Proposed solution.....	12
5	Proposed Design/ Model.....	13
5.1	High Level Diagram (if applicable).....	13
5.2	Low Level Diagram (if applicable).....	13
5.3	Interfaces (if applicable).....	13
6	Performance Test.....	14
6.1	Test Plan/ Test Cases.....	14
6.2	Test Procedure.....	14
6.3	Performance Outcome.....	14
7	My learnings.....	15
8	Future work scope.....	16

1 Preface

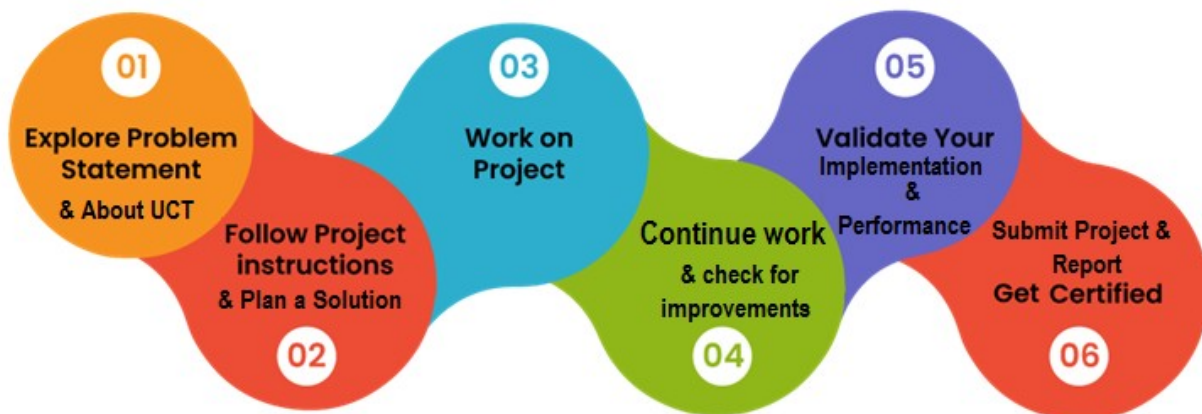
This report presents the work completed during a six-week industrial internship, from understanding a real marketplace problem to designing, implementing, and validating a working web application.

An internship is valuable because it connects academic concepts with practical delivery constraints. It develops the ability to translate user needs into software requirements, make trade-offs, test assumptions, and communicate technical decisions clearly.

The project addressed the fragmentation faced by customers and independent service providers. The Service Marketplace Platform gives providers a digital storefront and gives customers a clear path from discovery to booking and secure payment.

The opportunity provided through USC, The IoT Academy, and UCT enabled practical exposure to full-stack product development, API design, authentication, hosted payments, responsive interface design, and release validation.

The program was planned around problem understanding, solution design, implementation, integration, testing, documentation, and final review. Work was organized so that the main customer and merchant journeys were usable before secondary improvements were documented as future scope.



The internship strengthened my understanding of full-stack development, typed API contracts, third-party integrations, responsive design, validation, error handling, and professional technical documentation. It also improved my ability to evaluate what is production-ready and what requires further hardening.

I express my sincere gratitude to upskill Campus, The IoT Academy, UCT, and all mentors, reviewers, instructors, and peers who supported the internship process and provided guidance during the project.

My message to juniors and peers is to build the complete user journey, not only isolated features. Understand the problem first, keep interfaces simple, validate every important flow, and document limitations honestly.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



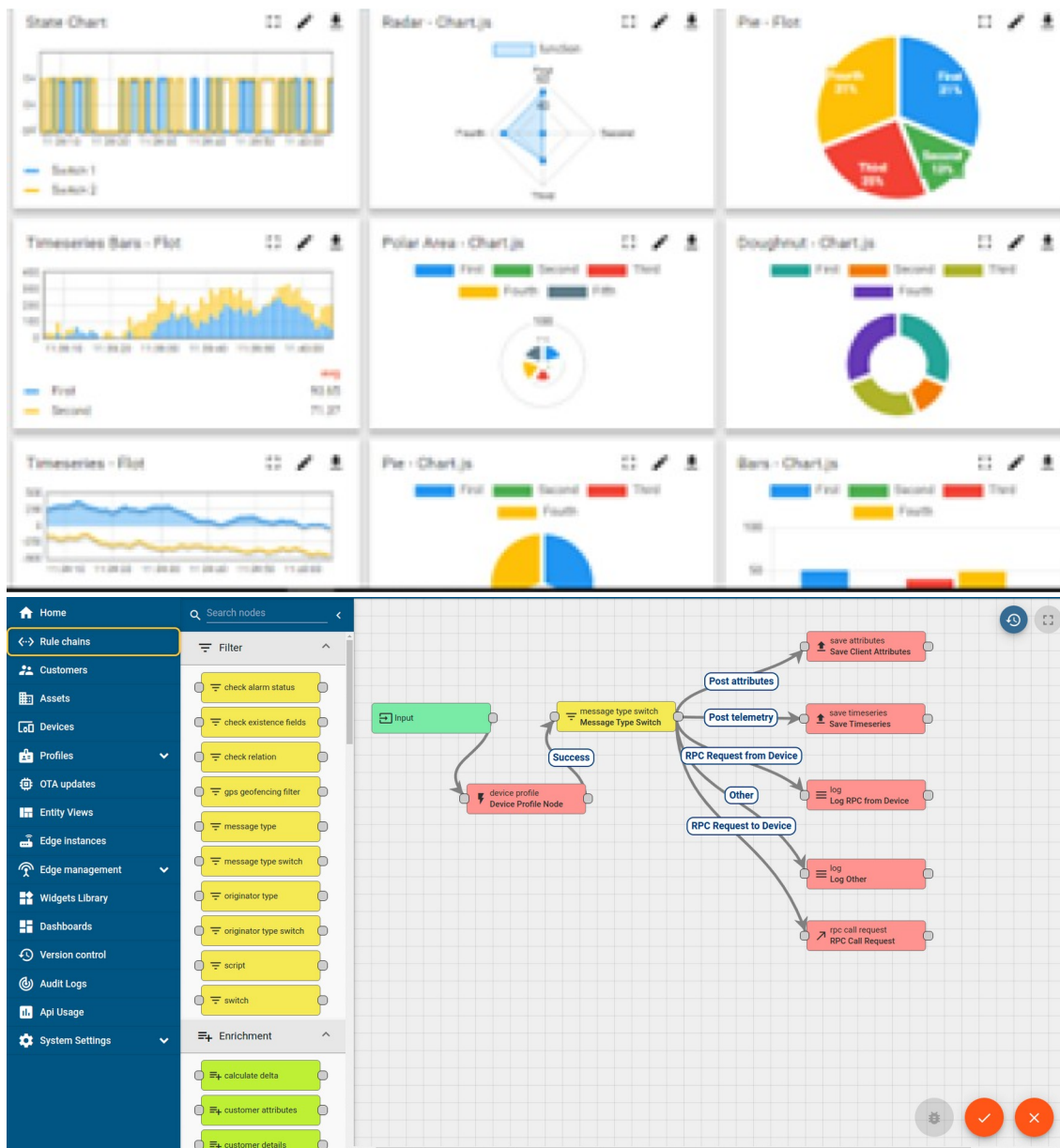
i. UCT IoT Platform (Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (**FACTORY WATCH**)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



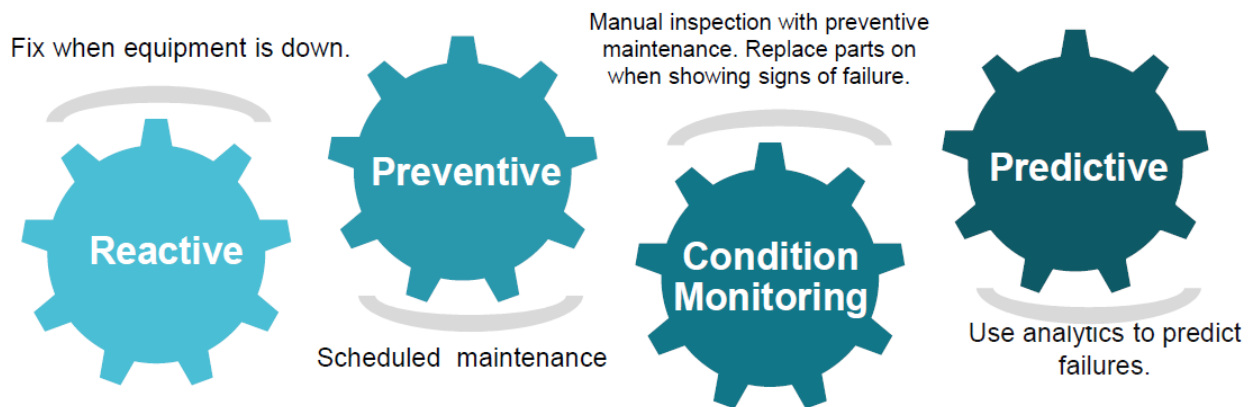


iii. based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

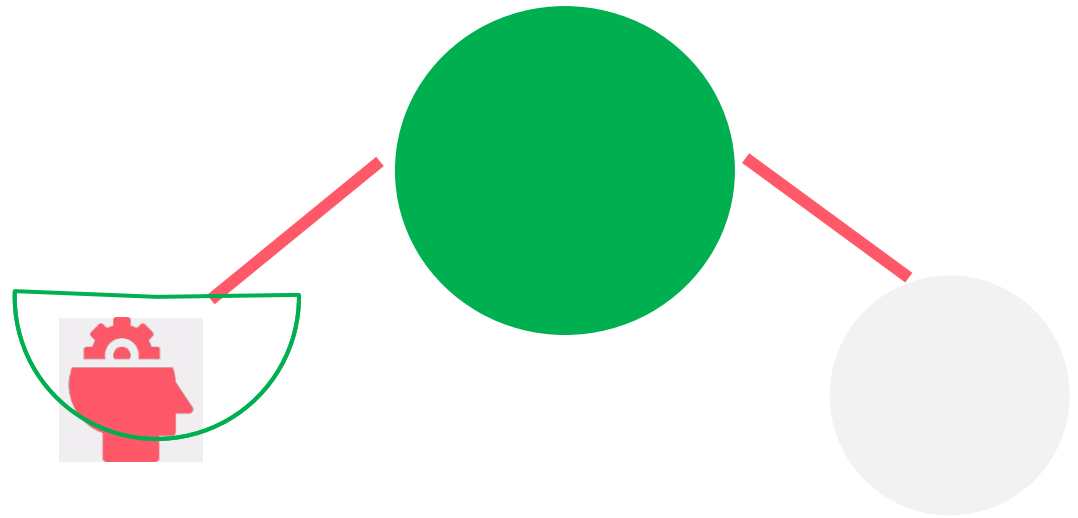
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

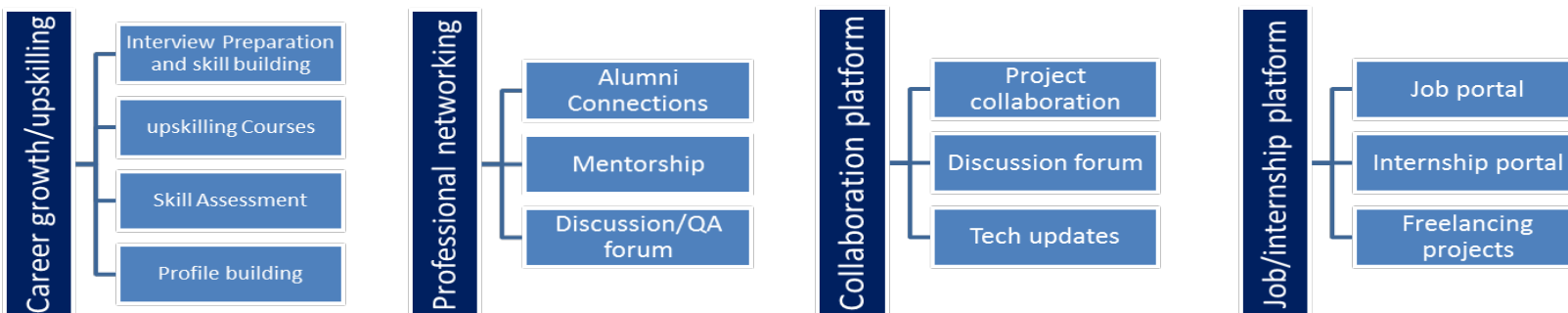
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling
in self paced manner
along-with additional
support services e.g.
Internship, projects,
interaction with Industry
experts, Career growth
Services

upSkill Campus
aiming to upskill 1
million learners in
next 5 year

[https://
www.upskillcampus.com/](https://www.upskillcampus.com/)



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

[1] OpenAPI Initiative, "OpenAPI Specification,"
<https://spec.openapis.org/oas/latest.html>

[2] Stripe, "Checkout and hosted payment pages,"
<https://docs.stripe.com/payments/checkout>

[3] Clerk, "Authentication documentation," <https://clerk.com/docs>

[4] React Documentation, <https://react.dev/>

[5] Project source repository, <https://github.com/MaheshKekan/Upskill-Campus>

2.6 Glossary

Terms	Acronym
API	Application Programming Interface

CRUD	Create, Read, Update, Delete
MVP	Minimum Viable Product
OAuth	Open authorization protocol used by identity providers
SaaS	Software as a Service
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator

3 Problem Statement

Independent service providers often depend on social media messages, phone calls, spreadsheets, or disconnected booking tools to present their services and manage appointments. Customers must search across multiple channels, ask for prices and availability manually, and use payment processes that may not provide a consistent record.

The project requirement was to design and implement a Service Marketplace Platform that connects customers with local providers through a clear, trustworthy digital experience. The platform needed to support provider discovery, service and price presentation, scheduling, secure payment handoff, order history, reviews, and basic merchant operations in a responsive web application.

- Provide a searchable directory of service providers and categories.
- Allow each provider to present a storefront with services, prices, availability, images, and reviews.
- Guide customers from service selection to appointment details and hosted Stripe checkout.
- Give merchants a simple way to create a storefront, add services, and review bookings.
- Use typed contracts, validation, authentication, and explicit error states to improve reliability.

4 Existing and Proposed solution

Existing alternatives include social media pages, local directory listings, phone-based booking, generic appointment tools, and direct messaging. These approaches can help providers become discoverable, but they often separate marketing, pricing, availability, payment, and order history across different channels. Customers may not know whether information is current, and providers may need to manually reconcile requests and payments.

The proposed solution is a multi-tenant marketplace with a public discovery layer and provider-specific storefronts. Customers can search by category or location, inspect service details, select a preferred date and time, and continue to a Stripe-hosted checkout session. Providers can create storefronts, add services, and inspect booking activity. Clerk is used for branded sign-in and sign-up experiences, while the API exposes validated marketplace resources through OpenAPI-generated client hooks.

The main value addition is a single, understandable journey from local discovery to confirmed booking. Clear prices, service descriptions, provider reviews, responsive layouts, secure hosted payment collection, and explicit loading/error/empty states reduce uncertainty for customers. The architecture also creates a foundation for future persistence, provider roles, availability calendars, notifications, and dispute management.

4.1 Code submission: <https://github.com/MaheshKekan/Upskill-Campus>

4.2 Report submission: <https://github.com/MaheshKekan/Upskill-Campus>

4.3

5 Proposed Design/ Model

The system was designed as a browser-based marketplace supported by a typed API. The frontend uses React, Vite, TypeScript, Tailwind CSS, and Wouter routing. The backend uses Express and TypeScript. OpenAPI definitions generate the client hooks used by the frontend, while Zod schemas validate request parameters and response shapes at the API boundary. Clerk handles account authentication and Stripe handles hosted payment collection.

5.1 High Level Diagram (if applicable)

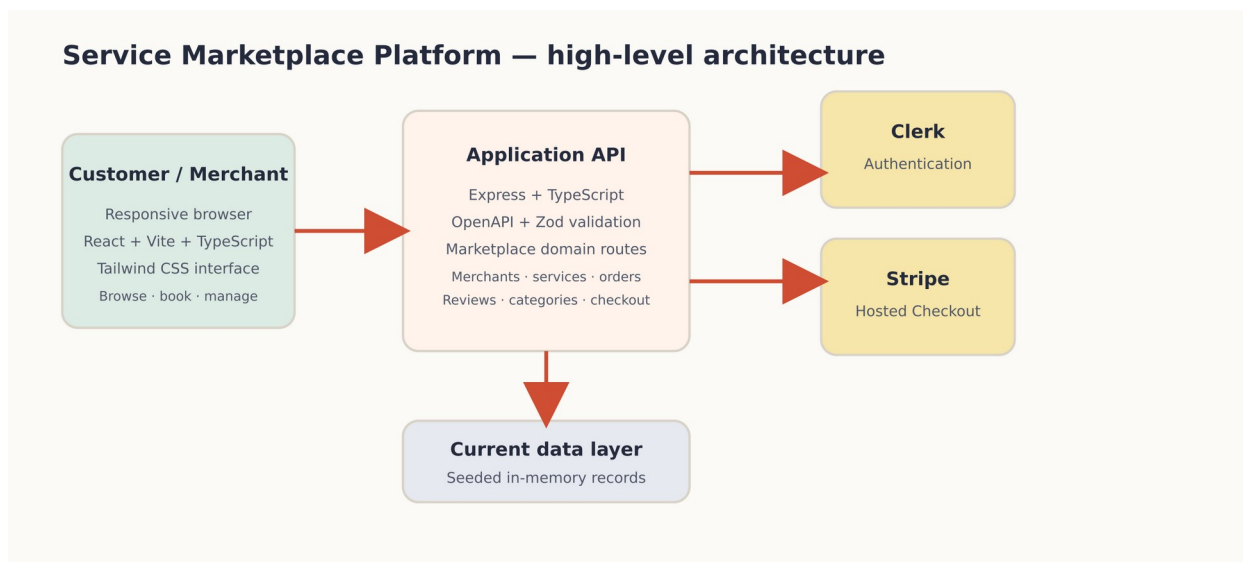
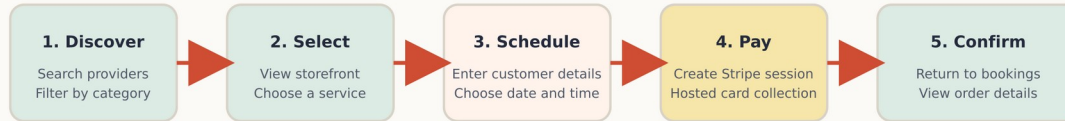


Figure 1: High-level architecture of the Service Marketplace Platform

5.2 Low Level Diagram (if applicable)

The detailed customer flow begins with a directory query, continues through storefront and service selection, collects appointment details, creates a validated order, and redirects the customer to hosted Stripe checkout. The order is then available through the booking history flow.

Customer booking flow



The API validates the merchant, service, customer details, and amount before creating the hosted payment session.

5.3 Interfaces (if applicable)

The main application interfaces are HTTP and JSON-based API endpoints consumed through generated TypeScript hooks. Important interfaces include GET `/api/categories`, GET `/api/merchants`, GET `/api/merchants/:slug`, GET `/api/merchants/:slug/services`, POST `/api/merchants`, POST `/api/merchants/:slug/services`, POST `/api/orders/checkout`, GET `/api/orders`, GET `/api/orders/:id`, and review endpoints. Clerk provides browser authentication, and Stripe is accessed through a server-side connector so payment details are collected by Stripe rather than by the application.

6 Performance Test

Performance and reliability were considered as product constraints rather than treated only as academic concerns. The implementation emphasizes typed request/response contracts, server-side validation, hosted payment collection, responsive layouts, and explicit loading, error, empty, and success states.

The main constraints were development time, safe payment handling, predictable API contracts, responsive rendering, and the need to keep the experience understandable for both customers and providers.

The design addresses these constraints by using OpenAPI-first code generation, Zod validation on the server, Clerk for authentication, Stripe-hosted checkout instead of collecting card data, reusable UI components, and a managed API/frontend workflow. The current build uses seeded in-memory records so the complete journey can be demonstrated quickly; this is explicitly documented as a persistence limitation.

The API server typecheck passed, the frontend typecheck passed, the API production build passed, and the frontend production build completed successfully. Runtime smoke checks returned valid category, merchant, and storefront-service responses. The live preview rendered the public homepage and branded Clerk sign-in page without browser errors that blocked rendering. No load test or production payment charge was performed during validation.

For a web marketplace, the relevant constraints are request latency, availability, data durability, payment correctness, and user trust. The current implementation handles validation and payment-session creation at the API boundary, but durability and scale require a database, webhook-driven payment confirmation, structured logging, and load testing before production use.

The recommendation is to add PostgreSQL persistence, indexes for merchant and order queries, Stripe webhook handling, automated email notifications, availability conflict checks, rate limiting, end-to-end tests, and a measured load-test baseline. These changes would turn the demonstrable MVP into a production-hardened marketplace.

6.1 Test Plan/ Test Cases

3. Verify checkout form validation and secure checkout-session creation. Result: Implemented and typechecked; a live payment transaction was not submitted during validation.

2. Verify storefront details include services, pricing, availability, and reviews. Result: Passed through the live storefront API response and rendered route.

1. Verify public category and provider discovery responses. Result: Passed through API smoke checks and live preview validation.

6.2 Test Procedure

The project was checked using generated API codegen, TypeScript typechecks for the API and frontend, production builds, workflow startup, browser preview screenshots, and curl-based API smoke requests for categories, merchants, and storefront details.

6.3 Performance Outcome

The production frontend build produced a 479.59 kB JavaScript bundle before gzip and a 142.03 kB gzipped JavaScript bundle, with a 105.70 kB CSS bundle before gzip. These figures establish an initial baseline. Formal latency, concurrency, availability, and payment-webhook benchmarks remain future work.

7 My learnings

This project improved my ability to work across the full software lifecycle. I learned how to translate a business problem into user journeys and API resources, maintain an OpenAPI-first contract, build reusable responsive interfaces, validate server inputs, integrate managed authentication, and connect a payment provider without handling raw card data. I also learned the importance of distinguishing a working MVP from a production-ready system: persistence, authorization, notifications, observability, testing, and operational safeguards must be planned explicitly. These skills will support future work as a full-stack developer because they combine implementation ability with product thinking, security awareness, and technical communication.

8 Future work scope

The following improvements are recommended for future development:

- Replace seeded in-memory records with PostgreSQL persistence and tenant-scoped data access.
- Add Stripe webhooks so successful payment events confirm orders reliably and support refunds or cancellations.
- Send customer receipts and merchant booking notifications after confirmed payment.
- Add provider availability calendars, conflict detection, rescheduling, cancellation, and dispute workflows.
- Protect merchant and admin operations with role-based authorization tied to Clerk identities.
- Move provider and service images to durable object storage with validation and moderation controls.
- Add automated unit, integration, end-to-end, accessibility, load, and security testing.
- Add analytics for searches, booking conversion, successful payments, and provider performance.

